COMSATS University Islamabad

Attock Campus

SUBMITTED BY: Asad Ali

REGISTRATION NO: FA24-BSE-047

ASSIGNMENT NO: 01

COURSE: Information Security Lab

SUBMITTED TO: Ms Ambreen Gul

DATE OF SUBMISSION: 28-02-2026

1. Code Screenshots

```python
# Only alphabetic characters are shifted, others remain unchanged
import string

# user defined function for encryption
def caesar_encrypt(text, shift):
    result = ""
    lower = string.ascii_lowercase
    upper = string.ascii_uppercase

    for ch in text:
        # for encrypting lowercase letters
        if ch.islower():
            result += lower[(lower.index(ch) + shift) % 26]

        # for encrypting upper letters

        elif ch.isupper():
            result += upper[(upper.index(ch) + shift) % 26]


         # for non alphabets chracter
        else:
            result += ch

     # for returning encrypted text
    return result

# user defined function for decryption

def caesar_decrypt(text, shift):
    result = ""
    lower = string.ascii_lowercase
    upper = string.ascii_uppercase
```

```python
    for ch in text:

        # decrypting lower case letters
        if ch.islower():
            result += lower[(lower.index(ch) - shift) % 26]

            # decrypting upper case letters
        elif ch.isupper():
            result += upper[(upper.index(ch) - shift) % 26]

            # decrypting non alphabetic chracters
        else:
            result += ch

    return result


# message which is going to be encrypted
message = "Ali lives in Pakistan. His age is 20. He studies at FAST University. Reg No: FA27-BSE-047"
# Shifting alphabets 6 places
shift = 6

encrypted = caesar_encrypt(message, shift)
print("Encrypted Text:")
print(encrypted)

decrypted = caesar_decrypt(encrypted, shift)
print("\nDecrypted Text:")
print(decrypted)
```

OUTPUT

```
Encrypted Text:
Gro robky ot Vgqoyzgt. Noy gmk oy 20. Nk yzajoky gz LGYZ Atobkxyoze. Xkm Tu: LG27-HYK-047

Decrypted Text:
Ali lives in Pakistan. His age is 20. He studies at FAST University. Reg No: FA27-BSE-047
```

2. Code detailed Line by Line Explanation:

Step 1: Import Required Module

```python
import string
```

Step 2: Define the Encryption Function

```python
def caesar_encrypt(text, shift):
```

This function takes:

- plain_text: the original message
- shift: number of letters to shift (6)

Step 3: Initialize Variables for Encryption

```
result = ""
    lower = string.ascii_lowercase
    upper = string.ascii_uppercase
```

- `result` stores the final encrypted message.
- Two separate alphabet strings are used to:
- Preserve lowercase letters
- Preserve uppercase letters

Step 4: Loop Through Each Character

```
for ch in text:
```

- The loop processes **each character one by one**.
- This allows checking whether the character is:
- Lowercase
- Uppercase

Step 5: Encrypt Lowercase Letters

```
for ch in text:
        if ch.islower():
```

```
            result += lower[(lower.index(ch) - shift) % 26]
```

- `lower.index(ch)` finds the position of the character in the lowercase alphabet.
- `shift` (6) is added to move the character forward.
- `% 26` ensures wrap-around from `z` back to `a`.
- The new shifted letter is added to `result`.

Step 6: Check for Uppercase Letters & Encrypt them

```
elif ch.isupper():
            result += upper[(upper.index(ch) - shift) % 26]
```

Step 7: Handle Non-Alphabetic Characters

```
else:
            result += ch
```

Step 8: Return Encrypted Result

```
    return result
```

Step 9: Define Decryption Function

```
def caesar_decrypt(text, shift):
```

- This function reverses the encryption process.
- Uses the same shift value.

Step 9: Initialize Variables

```
result = ""

    lower = string.ascii_lowercase

    upper = string.ascii_uppercase
```

- `result` stores the decrypted message.

Step 10: Loop Through Encrypted Text

```
    for ch in text:
```

Step 11: Decrypt Lowercase Letters

```
if ch.islower():
            result += lower[(lower.index(ch) - shift) % 26]
```

- The shift value is **subtracted** instead of added.
- `% 26` ensures wrap-around from `a` back to `z`.
- This restores the original lowercase letter.

Step 12: Decrypt Uppercase Letters

```
 elif ch.isupper():
            result += upper[(upper.index(ch) - shift) % 26]
```

- Same reverse logic for uppercase letters

Step 13: Preserve Special Characters

```
    else:
            result += ch
```

Step 14: Return Decrypted Text

```
    return result
```

- The original message is fully restored and returned.

Step 15: Input Message

```
message = "Ali lives in Pakistan. His age is 20. He studies at FAST University. Reg No: FA27-BSE-047"
```

Step 16: Shift Value

```
shift = 6
```

Step 17: Encrypt and Decrypt

```
encrypted = caesar_encrypt(message, shift)
print("Encrypted Text:")
print(encrypted)
decrypted = caesar_decrypt(encrypted, shift)
print("\nDecrypted Text:")
print(decrypted)
```

3. Security Analysis of Caesar Cipher:

- **Type of Cipher:** Classical substitution cipher using a fixed letter shift (6 in this program).
- **Strengths:** Easy to implement, preserves letter case, and keeps numbers, spaces, and symbols unchanged.
- **Weaknesses:** Small key space and vulnerable to brute-force or frequency attacks.
- **Purpose:** Mainly educational, to demonstrate basic encryption and decryption concepts.

Overall, the Caesar Cipher provides a simple way to encrypt and decrypt messages while maintaining readability for letters and preserving the original formatting of the text. Although it is not secure for protecting real-world data due to its predictable patterns, it is effective for learning the fundamentals of cryptography and understanding how encryption works in principle.